ECE 190 Final Report: Embedded Tamagotchi System

Melissa De La Cruz

April 2026

Abstract

This project presents the design and implementation of an embedded Tamagotchi-inspired system using an ESP32-S3 microcontroller. The system integrates an SPI display, user input buttons, and sensor-based interaction to simulate a virtual pet. This report covers hardware architecture, software design, PCB implementation, and key engineering challenges encountered.

Contents

1 Introduction

1.1 Project Motivation

An inspiration for this project is the Tamagotchi. I have been feeling a type of digital fatigue from social media, so I wanted to make a small device that can keep me entertained and away from the digital world.

I built this project in order to learn more about different communication protocols, PCB design, and how to work with sensor integration and embedded software. This project matters to me because it is a fun way for me to exercise through walking as well as a cute way to show my technical skills.

1.2 System Overview

My Embedded Tamagotchi system is an interactive virtual pet device built using an ESP32 microcontroller. The system simulates a digital pet that responds to user inputs and changes state over time. The main hardware components include the ESP32 for processing and control, a display module for the user interface, physical buttons for user input, and sensors such as an accelerometer for motion-based features.

The display presents the pet's current status, including attributes such as hunger, happiness, and energy, while the buttons allow the user to interact with the pet through actions like feeding, playing, and putting it to sleep. User interaction follows a simple loop: the user presses a button, the ESP32 processes the input, and the pet's state is updated and reflected on the display. This creates a responsive system where the pet reacts in real time to user actions.

Additionally, the system includes a pedometer feature that utilizes sensor data to track user movement.

Here is a list of all the things I used to create this project:

- **HiLetgo 0.96" SPI Serial 128*64 12864 Characters OLED LCD Display SSD1306:** Amazon
- **Memory Mode:** The `F` designation indicates a "Full Frame Buffer." This mode stores the entire display image in the microcontroller's RAM, enabling the use of `sendBuffer()` for faster, flicker-free updates compared to page-buffered modes.
- **Communication Protocol:** The 4W_HW_SPI suffix configures the library to use the 4-wire Hardware Serial Peripheral Interface (SPI).

This utilizes the microcontroller's dedicated SPI peripheral for high-speed data transmission, which is faster than "bit-banging." (Bit banging is a technique for serial communication in which the whole communication process is handled via software instead of dedicated hardware)

2 Communication Protocols

For the communication protocols, I wanted to dive into SPI and I2C in order to get the most experience as possible. I have a bit of experience with the Universal Asynchronous Receiver Transmitter (UART) from ECE 111 (Advanced Digital Design Project) and from my NASA internship with GPS testing, so working with these new communication protocols helped me better understand how to execute this project as well as possible. I also wanted to have the flexibility of using two different display protocols within the development process of this project.

2.1 SPI Implementation

2.1.1 What is SPI? How does it work?

SPI stands for Serial Peripheral Interface. It is an interface bus; a bus is a group of wires in a PCB that parallelly connect two devices together, commonly used to send data between microcontrollers and small peripherals such as shift registers, sensors, and SD cards, SD card reader modules, RFID card reader modules, and 2.4 GHz wireless transmitter/receivers.

There are two types of communication: serial and parallel. Here is an example of the transmission of the letter "C" in binary. For SPI, it is a serial connection.

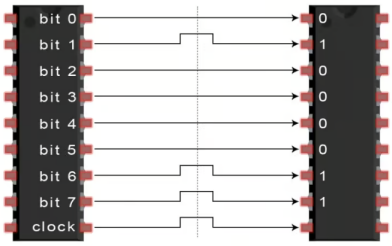

Figure 1: Parallel Communication

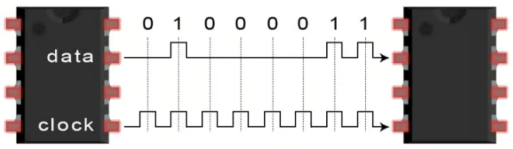

Figure 2: Serial Communication

A common serial port, such as UART, is called asynchronous because there is no control over when the data is sent. This can be a problem when two systems with slightly different clocks try to communicate with each other. To work around this problem, there is a start and stop bit added to each byte to help the receiver sync up to data as it arrives, and both the RX and TX must agree on the transmission speed (such as 9600 bits per second) in advance. The receiver samples the bits at very specific times (which are the arrows in Figure 1).

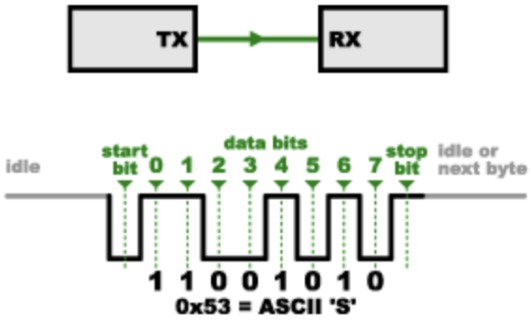

Figure 3: Serial Protocol with UART

SPI works a bit differently from UART. SPI is a synchronous data bus (a bus is a group of wires in a printed circuit board that parallelly connect two devices together), which means that it uses separate lines for data and a "clock" that keeps both sides in perfect sync.

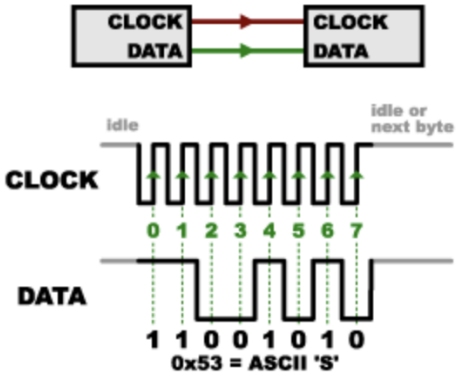

Figure 4: Serial Protocol with SPI

The clock is an oscillating signal that tells the receiver exactly when to sample bits on the data line during its rising (low to high) or falling edge(high to low). The datasheet specifies which edge to use for receiving data. When the receiver detects that edge, it will immediately look at the data line to read the next bit (dotted lines in Figure 2). Because the clock is sent along with the data, specifying the speed isn't important, although devices will have a top speed at which they can operate.

For the process of the transmitter and how it sends data to the receiver, in SPI, only one side generates the clock signal (usually called CLK or SCK for serial clock). The side that generates the clock is called the master/controller (ESP32-S3) and the other side is called the slave/peripheral (display). There is always one controller, which is the microcontroller. I used the simplest configuration of the SPI, which is one controller to one peripheral. One controller can have multiple peripherals.

2.1.2 Why SPI?

One reason that SPI is so commonly used is that the receiving hardware can be a simple shift register (a shift register is a group of flip-flops connected in series to store and shift multiple bits of data either left or right with clock pulses). This is a simpler and cheaper piece of hardware than the UART. I chose SPI because it's faster than I2C and gives me better control over timing since it's synchronous. It also works well with displays like the SSD1306.

The MISO pin was not used because the OLED display was treated

as a write-only peripheral. This means that the ESP32-S3 only needed to send commands and pixel data to the display using the MOSI line. The OLED did not need to send data back to the microcontroller, so the MISO line, which is normally used for slave-to-master communication in SPI, was unnecessary.

2.1.3 Pin Implementation (Use of SPI)

For the pin implementation, I used Serial Clock (SCK), Master Out Slave In (MOSI), Chip Select (CS), Data/Control (DC), and Reset Signal (RES).

For my project, SPI was implemented in the display driver for the SSD1306 OLED. In software, the SPI-related pins were explicitly assigned as SCK = 11, MOSI = 10, RES = 9, DC = 8, and CS = 7 in the display module (display.cpp(line 5)). The OLED was configured using the U8g2 4-wire hardware SPI constructor, which used the chip select, data/command, and reset pins (display.cpp (line 46)). During initialization, the ESP32-S3 started the SPI bus with

```
SPI.begin(PIN_SCK, -1, PIN_MOSI, PIN_CS)
```

showing that the project used the clock line, a transmit data line, and chip select, while omitting MISO because the display did not need to send data back to the microcontroller (display.cpp(line 199)).

2.1.4 How the SPI display connection was used

After initialization, the microcontroller updated display variables such as temperature, time, and date from the main application logic (main.cpp (line 158), main.cpp (line 234)). The display module then drew the user interface elements, including the pet sprite, status icons, and text labels, before transmitting the full frame buffer to the OLED with u8g2.sendBuffer() (display.cpp (line 420), display.cpp (line 462)). This demonstrates how SPI was not only wired physically, but also used in the code to continuously send display data from the ESP32-S3 to the OLED.

2.2 I2C Implementation

2.2.1 What is I2C?

I2C (Inter-Integrated Circuit) is a synchronous communication protocol that uses only two wires: Serial Data (SDA) and Serial Clock (SCL). Unlike SPI, which requires multiple dedicated lines, I2C allows multiple devices to

share the same communication bus, making it efficient for connecting several peripherals.

In this project, I2C was used to interface with the MPU6050 accelerometer. The ESP32-S3 acted as the master device, while the MPU6050 operated as the slave. Communication occurred over the SDA and SCL lines, with the microcontroller initiating all data transfers. The primary advantage of I2C in this system was its simplicity and use of less data lines compared to SPI. Since the accelerometer only required periodic data reads, I2C provided a clean and efficient solution without the need for high-speed data transfer.

3 Hardware Design

3.1 Microcontroller Selection (ESP32-S3)

The ESP32-S3 was selected as the main microcontroller due to its strong processing capabilities, extensive GPIO availability, and built-in wireless features such as Wi-Fi and Bluetooth. I used a custom ESP-32-s3 provided by Envision at UCSD, which makes the project more friendly to rebuild for students that are more curious about embedded systems.

One of the key advantages of the ESP32-S3 is its compatibility with a wide range of libraries and peripherals, which simplified integration with the display, sensors, and input devices. Additionally, its support for multiple communication protocols such as SPI and I2C made it ideal for handling different hardware components within the system.

The ESP32-S3 also offers sufficient computational power to manage real-time user interaction, display rendering, and sensor processing simultaneously, making it well-suited for this embedded application.

3.2 Display Tradeoffs (OLED vs TFT)

Two display options were considered for this project: OLED and TFT displays. OLED displays offer lower power consumption, higher contrast, and simpler interfacing, making them ideal for small embedded systems. In contrast, TFT displays provide full color and larger screen sizes but require more complex drivers and higher power consumption.

For this project, an OLED display was selected due to its simplicity, lower power requirements, and ease of integration with the ESP32-S3. Since the application primarily required simple graphics and text rendering, the OLED display provided a sufficient and efficient solution.

The TFT display will still be considered for a future rendition of this project, since it has a touchscreen feature if I want to go more into the firmware for this project or have more complex graphics.

3.3 Sensors

3.3.1 Accelerometer and Gyroscope (MPU6050)

The MPU6050 was used in this project to enable motion-based interaction with the Tamagotchi. The MPU6050 is an inertial measurement unit (IMU) that contains both a 3-axis accelerometer and a 3-axis gyroscope. The accelerometer measures linear acceleration along the x, y, and z axes, while the gyroscope measures rotational motion around those axes. For this project, the accelerometer data was the main sensor data used for detecting walking and shaking motions.

The MPU6050 communicates with the ESP32-S3 using the I2C protocol. I2C was useful for this sensor because it only requires two main communication lines: SDA for data and SCL for the clock. In my implementation, the I2C bus was initialized using `Wire.begin(16, 15)`, where GPIO 16 was used for SDA and GPIO 15 was used for SCL. After the sensor was detected, it was configured with an accelerometer range of `MPU6050_RANGE_8_G`, a gyroscope range of `MPU6050_RANGE_500_DEG`, and a filter bandwidth of `MPU6050_BAND_21_HZ`. These settings helped make the sensor usable for detecting larger body movements while reducing some high-frequency noise.

The sensor was read through the `imuRead()` function, which collected acceleration and gyroscope events from the MPU6050. The acceleration values were stored as `ax`, `ay`, and `az`, while the gyroscope values were stored as `gx`, `gy`, and `gz`. These values were then used by the pedometer and shake detection algorithms in the main program loop.

3.3.2 Use Case: Step Detection

One major use case for the MPU6050 was the step detection feature. This feature was connected to the Tamagotchi's play mode, so the pet would respond when the user physically moved with the device. Before step detection began, the program performed a calibration period where the user stood still for three seconds. During this time, the ESP32-S3 collected 100 acceleration samples and calculated an average baseline. This baseline represented the normal acceleration level when the device was not moving significantly.

For step detection, the program calculated the L1 norm of the acceleration data using the equation:

$$|a_x| + |a_y| + |a_z|$$

This combined the acceleration from all three axes into one value that represented the total motion detected by the sensor. The pedometer stored recent acceleration values in a rolling buffer and calculated a moving average to smooth out the data. The moving average was then compared to the calibrated baseline. If the difference was greater than a threshold, the program counted it as a step.

To avoid counting the same movement multiple times, the pedometer used a `stepDetected` flag. Once a step was detected, the flag prevented another step from being counted until the motion dropped back below a lower level. This made the step detection more stable and reduced false repeated counts. In the Tamagotchi game logic, every detected step increased the step count, and every 15 steps increased the pet's happiness level. This connected real-world movement to the virtual pet's emotional state.

3.3.3 Use Case: Shake Detection

The MPU6050 was also used for shake detection during feed mode. Instead of looking for steps, the shake detector looked for sudden changes in acceleration between consecutive sensor readings. The algorithm compared the current acceleration values to the previous acceleration values and calculated the change in motion. If the total change was greater than a threshold for multiple samples, the program recognized the motion as a shake.

This shake interaction was used to feed the Tamagotchi. When the user entered feed mode and shook the device, the program increased the pet's fullness level. A cooldown time was included so that one shake motion would not accidentally trigger the feeding action multiple times in a row. This made the interaction feel more intentional and helped prevent false detections.

3.3.4 Why This Sensor Was Chosen

The MPU6050 was chosen because it is compact, inexpensive, and commonly used in embedded systems projects. Since it includes both an accelerometer and a gyroscope, it provides more motion data than a basic accelerometer alone. It also works well with the ESP32-S3 because it communicates through I2C, which saves GPIO pins compared to using many separate signal lines.

Another reason the MPU6050 was a good choice was library support. The Adafruit MPU6050 library made it easier to initialize the sensor, configure its measurement ranges, and read acceleration values in the Arduino/PlatformIO environment. This allowed more time to be spent on designing the Tamagotchi interactions, such as walking, playing, and feeding, instead of writing low-level sensor communication code from scratch.

Overall, the MPU6050 helped make the project more interactive by allowing physical motion to directly affect the pet's behavior. Walking with the device increased the pet's play/happiness level, while shaking the device in feed mode increased its fullness level.

3.4 User Input

User input was implemented using three physical push buttons. These buttons allowed the user to interact with the Tamagotchi without needing a touchscreen or keyboard input. Each button controlled one main mode of the pet: feed, play, and bed. In the code, the feed button was assigned to GPIO 4, the bed button was assigned to GPIO 5, and the play button was assigned to GPIO 6.

3.4.1 Button Design

The three-button design was chosen because it made the interface simple and easy to understand. Each button had a direct purpose:

- The **Feed** button enters or exits feed mode.
- The **Play** button enters or exits play mode.
- The **Bed** button enters or exits sleep mode.
- All **three buttons** pressed simultaneously would reset the device.

This design made the Tamagotchi feel more like a small handheld device, where each physical button controlled a specific action. In the display code, these buttons were defined as `BTN_FEED`, `BTN_BED`, and `BTN_PLAY`. The main loop repeatedly called `check_buttons()` so that button presses could update the current state of the pet.

3.4.2 Pull-up vs. Pull-down Implementation

The buttons were configured using the ESP32-S3's internal pull-up resistors. This was done with `pinMode(..., INPUT_PULLUP)` for each button. With a pull-up configuration, the input pin normally reads `HIGH` when the button is not pressed. When the button is pressed, the pin is connected to ground and reads `LOW`.

This means the buttons are active-low. Instead of checking for a transition from `LOW` to `HIGH`, the code checks for a transition from `HIGH` to `LOW`. This transition represents the moment the user presses the button. Using internal pull-up resistors also simplified the hardware wiring because external pull-down resistors were not needed.

3.4.3 Debouncing Strategy

A common issue with mechanical push buttons is bouncing. When a button is pressed, the metal contacts inside the button may rapidly connect and disconnect for a very short time before settling. If this is not handled, one physical press can accidentally be read as multiple presses.

In this project, the button logic used edge detection to reduce repeated triggering. The code stored the previous state of each button using variables such as `lastPlayState`, `lastFeedState`, and `lastBedState`. Each time `check_buttons()` ran, the current button state was compared to the previous state. A button action only occurred when the previous state was `HIGH` and the current state was `LOW`, meaning the code detected the first moment of a button press.

This approach prevented the mode from toggling continuously while the user held the button down. For example, if the play button was pressed, the code toggled `playMode` only once when the press was first detected. The same strategy was used for feed mode and sleep mode. Although this is not a full time-based debounce system, it worked as a simple software debounce method for this project by preventing repeated actions from a single held press.

3.4.4 Mode Control

The buttons were also designed so that the Tamagotchi would only be in one major interaction mode at a time. When the play button was pressed, feed mode and sleep mode were turned off before play mode was toggled. When the feed button was pressed, play mode and sleep mode were turned

off before feed mode was toggled. When the bed button was pressed, both play and feed modes were turned off before sleep mode was toggled.

This helped keep the program logic organized because the pet could not accidentally be sleeping, feeding, and playing at the same time. The button inputs therefore acted as a simple **state-control system** for the Tamagotchi.

3.5 Power System

3.6 Display Rendering

The display rendering system was responsible for drawing the Tamagotchi interface on the OLED screen. The project used the U8g2 graphics library with an SSD1306 128x64 OLED display. To interface the software with the physical hardware, the library uses a specific hardware SPI constructor, which allowed the ESP32-S3 to send graphics data to the screen using the SPI communication protocol:

```
U8G2_SSD1306_128X64_NONAME_F_4W_HW_SPI
```

. This configuration string defines several critical parameters:

- **Controller & Resolution:** It specifies the `SSD1306` driver chip and a `128x64` pixel resolution.
- **Memory Mode:** The `F` designation indicates a "Full Frame Buffer." This mode stores the entire display image in the microcontroller's RAM, enabling the use of `sendBuffer()` for faster, flicker-free updates compared to page-buffered modes.
- **Communication Protocol:** The 4W_HW_SPI suffix configures the library to use the 4-wire Hardware Serial Peripheral Interface (SPI). This utilizes the microcontroller's dedicated SPI peripheral for high-speed data transmission, which is faster than "bit-banging." (Bit banging is a technique for serial communication in which the whole communication process is handled via software instead of dedicated hardware)

The constructor arguments

```
(U8G2_R0, PIN_CS, PIN_DC, PIN_RES)
```

further define the screen's landscape orientation and the specific digital pins used for Chip Select (CS), Data/Command (DC), and Reset (RES) control.

3.6.1 Graphics Libraries Used

The U8g2 library was used because it provides built-in functions for drawing text, pixels, frames, and bitmap images on small monochrome displays. Since the OLED display only supports black-and-white pixels, the graphics were designed using simple shapes and binary bitmap patterns.

The general rendering process followed the same structure for each screen. First, `u8g2.clearBuffer()` cleared the display buffer in memory. Then, drawing functions such as `u8g2.drawFrame()`, `u8g2.drawStr()`, `u8g2.drawPixel()`, `u8g2.drawBox()`, and `u8g2.drawXBMP()` were used to build the screen layout. Finally, `u8g2.sendBuffer()` sent the completed buffer to the OLED display. This means the screen was prepared in memory first and then transferred to the physical display all at once.

3.6.2 Bitmap Design

Several custom bitmap graphics were created to represent the Tamagotchi's status and personality. The main pet sprite was stored as a 32-pixel wide binary shape called `ORANGE_SHAPE`. Each bit in the sprite represented whether a pixel should be turned on or off. The program looped through each row and column of this bitmap and used `u8g2.drawPixel()` to draw the pet on the screen.

Smaller 8x8 bitmap icons were also used for different status meters. A heart bitmap represented the play level, a thunder bolt represented the energy level, a star represented overall happiness, and an ice cream bitmap represented fullness. These icons were drawn using `u8g2.drawXBMP()`, which made it easier to repeat the same icon multiple times for meter-style displays.

The pet sprite was also designed to change depending on the Tamagotchi's state. In normal and play mode, the pet was drawn with open eyes and a small smile. In bed mode, the same sprite function was used, but the eyes were drawn as closed rectangles to make the pet look like it was sleeping. In feed mode, the pet's scale changed based on the fullness level, making the sprite grow larger as the pet became more full.

3.6.3 UI Layout

The OLED screen resolution was 128x64 pixels, so the interface had to be compact and organized. Each screen used a border frame around the display to create a consistent layout. Text was drawn using a small font, `u8g2_font_6x10_tr`, so that status information and labels could fit within the limited screen space.

The home screen displayed the most important information at a glance. The top row showed the current time, date, and temperature. The middle area displayed the pet and the happiness meter, while the bottom row showed button labels for Feed, Play, and Bed. This made the home screen act as the main dashboard for the virtual pet.

Different modes used different layouts. Play mode displayed the step count, play hearts, and the pet animation. Feed mode displayed the instruction `"Shake to feed!"`, a fullness meter, and a pet sprite that changed size based on fullness. Bed mode displayed the sleeping pet, small `z` characters, and an energy meter. The function `display_Home()` acted as the main display dispatcher by checking the current mode and calling the correct rendering function.

3.6.4 Animation

Simple animation was added by changing the pet's position over time. The function `updatePetAnimation()` adjusted the pet's vertical offset when the pet was walking. In play mode, the pet also shifted slightly left and right using a time-based value from `millis()`. These small changes made the OLED feel more interactive even though the display was monochrome and low resolution.

Overall, the display rendering system combined text, custom bitmaps, simple animation, and mode-based layouts to create a complete Tamagotchi-style user interface on a small SPI OLED screen.

3.7 Pedometer Algorithm

The pedometer feature was implemented using data from the MPU6050 accelerometer. The system detects steps by analyzing changes in acceleration and identifying peaks that exceed a predefined threshold.

When the acceleration magnitude crosses this threshold, it is counted as a step. This method provides a simple and computationally efficient way to approximate step count in real time.

However, this approach has limitations, including sensitivity to noise and false positives from non-walking movements. Despite these limitations, the algorithm provides a functional and lightweight solution suitable for this embedded system.

4 Software Design

4.0.1 Wi-Fi Implementation

Wi-Fi was used in this project to make the Tamagotchi system more dynamic by allowing the ESP32-S3 to retrieve real-time information from the internet. In the code, the Wi-Fi library is included using `WiFi.h`, and the ESP32-S3 connects to a local Wi-Fi network during the `setup()` function. A connection timeout was also implemented so that the device would not get stuck forever if Wi-Fi was unavailable. If the connection succeeds, the system prints a confirmation message and continues with internet-based features; if it fails, the project continues running offline.

The first major use of Wi-Fi was time synchronization. After the ESP32-S3 connects to Wi-Fi, the code calls `configTime()` with the NTP server `pool.ntp.org`. NTP stands for Network Time Protocol, and it allows the microcontroller to get the current time from the internet instead of requiring an external real-time clock module. In the main loop, the code checks whether Wi-Fi is connected and then calls `getLocalTime()` to retrieve the current time. This time is converted from 24-hour format to 12-hour format and then passed into the display module using `display_setTime()` and `display_setDate()`. This allows the OLED screen to show the current time and date on the home screen.

The second major use of Wi-Fi was retrieving weather data. The project uses the `HTTPClient` library to send an HTTP GET request to the OpenWeatherMap API. The API request includes a ZIP code and requests the temperature in Fahrenheit. Once the ESP32-S3 receives a successful response, the JSON data is parsed using the ArduinoJson library. The temperature value is extracted from the JSON response and passed into the display module using `display_setTemp()`. The OLED then displays the current temperature in the top-right corner of the home screen.

To make the system more reliable, the code also includes reconnect behavior. If Wi-Fi is disconnected during the main loop, the ESP32-S3 periodically attempts to reconnect instead of stopping the rest of the program. Weather updates are also limited using a timer so that the device does not continuously call the API every loop iteration. In this implementation, the weather update interval was set to 60 seconds.

4.1 Data Persistence

Data persistence was implemented so that the Tamagotchi's state would not reset every time the ESP32-S3 was restarted or powered off. Without persistence, values such as step count, play level, fullness, and energy would return to zero whenever the device rebooted, which would make the pet feel less continuous and interactive.

To solve this, the project used the ESP32-S3's non-volatile storage through the `Preferences` library. Non-volatile storage means that the saved data remains stored even when the microcontroller loses power. In the code, a `Preferences` object is created and opened under the namespace `"tama-pet"`. This namespace acts like a storage category for the Tamagotchi's saved values.

When the program starts, the function `loadPetState()` reads the saved values from memory. The values loaded include `stepCount`, `playHearts`, `fullnessLevel`, and `energyLevel`. If no previous value exists, the code uses a default value of zero. The loaded values are also clamped to their expected ranges, which helps prevent invalid values from being used if memory contains unexpected data.

During the main loop, the pet's state changes based on user interaction. For example, walking in play mode can increase `stepCount` and `playHearts`, shaking the device in feed mode can increase `fullnessLevel`, and leaving the pet in bed mode can increase `energyLevel`. After these updates, the function `savePetStateIfChanged()` checks whether each value has changed. If a value is different from the last saved value, it writes the new value to non-volatile storage using `preferences.putInt()`.

This approach avoids unnecessary writes by only saving values when they change. This is important because flash memory has a limited number of write cycles, so reducing repeated writes helps improve reliability. Overall, persistence allowed the Tamagotchi to maintain its progress across resets and power cycles, making the pet behave more like a continuous virtual companion rather than restarting from the beginning each time.

Preferences is included and created in main.cpp (line 10), saved values are loaded in main.cpp (line 64), changed values are written in main.cpp (line 81), and the save function is called during the main loop in main.cpp (line 312).

5 PCB Design

5.1 Schematic Design

For the tools used, I used KiCad due to previous experience with the software from a past class.

Here is the schematic created:

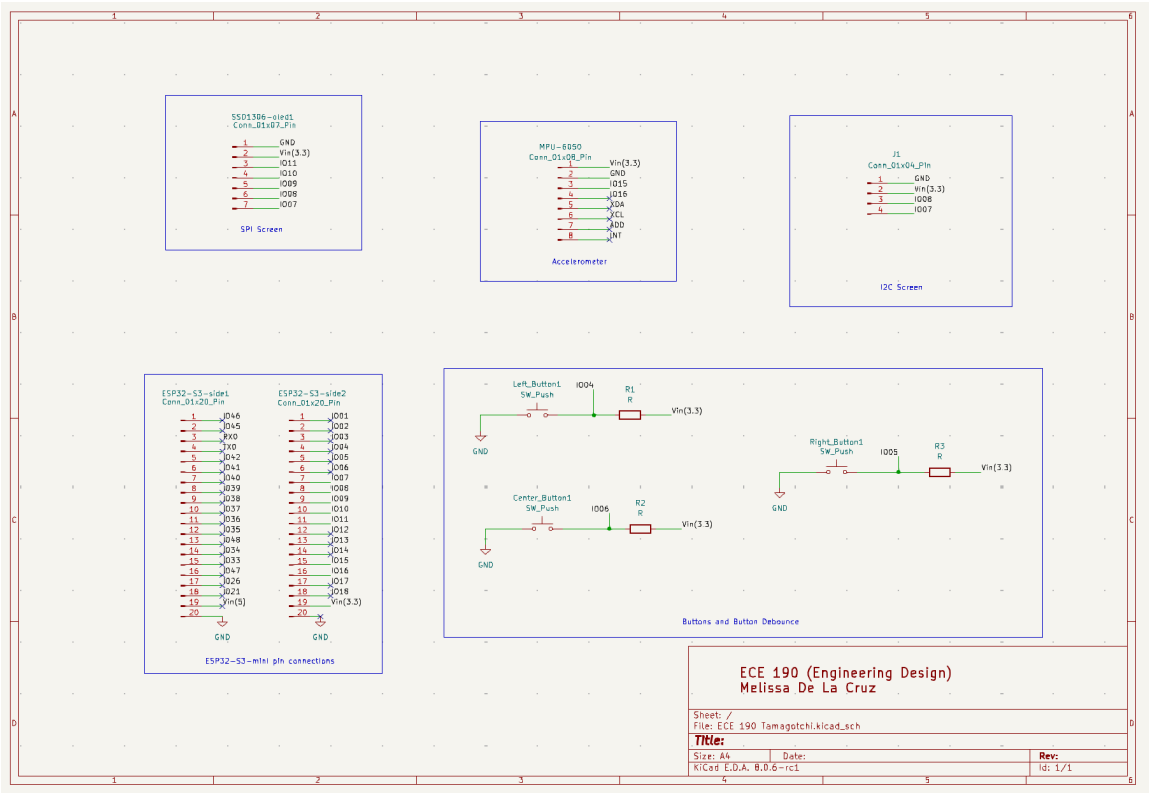

Figure 5: Custom PCB Schematic for Tamagotchi

I had to choose all the footprints for each component in order to fit all components on the PCB accurately. I mostly used generic pin connection footprints for the display, accelerometer, and the ESP-32-S3.

5.2 Component Selection and Important Concepts

I originally had resistors for the switches in order to

5.3 Manufacturing Considerations

For the manufacturing, I used JLCPCB since it was recommended to me as being a good PCB manufacturing company.

6 Testing and Results

6.1 Hardware Testing

6.2 Software Validation

6.3 System Performance

6.4 User Interaction Results

7 Challenges and Solutions

7.1 Hardware Issues

7.2 Software Bugs

7.3 Integration Challenges

7.4 Lessons Learned

8 Future Work

8.1 Improvements

8.2 Hardware Enhancements

9 Conclusion

10 Acknowledgments

11 Works Cited

- https://www.arkco-sales.com/articles/hmi-tft-lcd-oled
- https://learn.sparkfun.com/tutorials/serial-peripheral-interface-spi/all
- https://www.reddit.com/r/embedded/comments/139cmhs/can_you_please_explainlikeimfive_the_difference/?solution=51fa3e790346d92751fa3e790346d9js_challenge=1&token=bbbe4bf1c9a2b5160829c4be34da5861c4a78600c3d93e3f6e204059c3e
- https://cdn-shop.adafruit.com/datasheets/SSD1306.pdf
- https://www.circuitbasics.com/basics-of-the-spi-communication-protocol/
- https://circuitdigest.com/article/introduction-to-bit-banging-spi-communication-what-is-bit-banging-understanding-software-based-communication